

Week 3. CNI Overview & Cloud Native Networking Landscape

1. CNI란?

CNCF(Cloud Native Computing Foundation)의 프로젝트 중 하나인 CNI는 컨테이너 간의 네트워킹을 제어할 수 있는 플러그인을 만들기 위한 표준이다. 쿠버네티스에서는 Pod 간의 통신을 위해서 CNI를 사용한다. CNI의 관심사는 **1. 컨테이너에 네트워크를 연결하는 것, 2. 컨테이너 삭제 시 할당된 리소스를 회수하는 것**이다.

CNI가 표준 인터페이스로서 제공하는 것

CNI는 '이렇게 호출하면 이렇게 응답해야 한다'는 계약(contract)을 제공한다.

- 실행 규약
: 각 CNI 플러그인은 실행 파일(executable) 형태이며, 런타임이 이를 호출한다.
- 표준 오퍼레이션
: **ADD** , **DEL** , **CHECK** , **VERSION** 네가지 동작을 플러그인이 구현해야 한다.
- 입출력 규약
: 설정은 **stdin(JSON)** 으로 전달, 결과는 **stdout(JSON)** 으로 반환해야 한다.
- 파라미터 전달 방식
: 컨테이너 ID, 네트워크 네임스페이스 경로, 인터페이스 이름 등을 환경변수로 전달한다.
- 플러그인 체이닝
: 여러 플러그인을 순서대로 실행해 기능을 조합한다.

즉, CNI 덕분에 쿠버네티스는 어떤 네트워크 솔루션을 쓸지 몰라도 되고, 네트워크 벤더는 쿠버네티스의 내부 구조를 몰라도 동일한 방식으로 플러그인을 만들 수 있다.

2. CNI Plugin의 동작 흐름

week2에서 다뤘던 Pod 생성 흐름을 CNI 관점에서 좀 더 구체화하면 아래와 같다.

```
kubelet
├─ CRI (containerd/CRI-O)에 Pod Sandbox 생성 요청
│   └─ Pod용 Network Namespace 생성
│       └─ CRI가 CNI 플러그인 실행 파일을 호출
```

- └ 표준 입력(JSON)으로 설정 전달(어떤 netns인지, 어떤 인터페이스명인지 등...)
- └ 플러그인이 veth pair 생성, 한쪽을 netns에 배치
- └ IPAM 플러그인을 호출하여 IP 할당
- └ 라우팅 설정, 결과를 표준 출력(JSON)으로 반환

CNI 플러그인의 종류

종류	예시	역할
Main / Interface 플러그인	bridge, ptp, macvlan, host-device	실제 네트워크 인터페이스를 만들고 netns에 연결
IPAM 플러그인	host-local, dhcp, static	인터페이스에 할당할 IP 주소를 결정
Meta 플러그인	portmap, bandwidth, tuning, firewall	위 플러그인 실행 후 추가 기능 부여

Primary CNI(Calico, Cilium, Flannel 등)는 보통 이 세 종류를 하나의 통합된 솔루션으로 자체 제공한다.

3. IPAM이 왜 필요한가?

IPAM(IP Address Management)은 CNI 플러그인 실행 시 Pod에 어떤 IP를 줄 지 담당하는 서브 플러그인이다.

IPAM이 없다면?

- 여러 노드에서 동시에 Pod가 생성될 때 IP가 중복 할당될 수도 있고...
- 노드마다 별도로 IP를 관리하면 클러스터 전체의 IP 대역(CIDR)이 충돌할 수도 있고...
- Pod 삭제 시 IP를 회수하지 않으면 IP가 고갈될 수도 있고...
- 어떤 IP가 어떤 노드/파드에 속하는지 추적이 안되면 라우팅 정보를 구성할 수가 없고...

IPAM의 역할

- 클러스터 전체 Pod CIDR을 노드별 서브넷으로 분할
- 노드 내에서 개별 Pod에 중복 없는 IP 할당
- Pod 삭제 시 IP 반납 및 재사용 관리
- 필요 시 외부 IPAM 시스템(DHCP, 클라우드 VPC 서브넷)과 연동

4. Primary CNI Overview

Primary CNI란 Pod 네트워크의 기본 연결성(connectivity)을 책임지는 CNI를 말한다.

CNI	핵심 기술	대표적인 특징
Flannel	VXLAN(기본) / host-gw 등	가장 단순한 구조, 순수 네트워킹 연결에 집중
Calico	BGP 기반 L3 라우팅 (기본: 오버레이 없음) + IP/IPV6/VXLAN 오버레이 옵션	라우팅 중심, 강력한 NetworkPolicy, 데이터플레인 선택 가능 (Linux/eBPF/Windows/VPP)
Cilium	eBPF 기반 데이터플레인	커널 레벨 성능, kube-proxy 완전 대체 가능, L3~L7 identity 기반 정책

5. Flannel / Calico / Cilium 아키텍처 차이

Flannel

- 각 노드에 `flanneld` 에이전트가 상주하며, 노드별로 서브넷 lease를 할당받음
- 노드 간 통신은 기본적으로 VXLAN 오버레이로 캡슐화
- 설정 저장소로 etcd 또는 Kubernetes API(kube subnet manager 모드)를 사용
- 네트워크 연결에만 집중하며, NetworkPolicy는 자체 지원하지 않음
- 구조가 단순해서 학습/테스트 클러스터에 자주 쓰임

Calico

- 오버레이 없이 순수 L3 라우팅을 기본으로 지향 - 각 노드가 vRouter 역할을 하고 BGP로 경로를 전파
- 정책 엔진인 Felix가 각 노드에서 Kubernetes NetworkPolicy + Calico 자체 정책을 강제
- 필요 시 IP/IPV6 또는 VXLAN 오버레이로도 동작 가능 (네트워크 환경 제약이 있을 때)
- 데이터플레인을 선택 가능: 표준 Linux, eBPF, Windows HNS, VPP 등
- CNI 플러그인이 자체 IPAM(블록 단위 동적 할당)을 포함

Cilium

- eBPF를 커널에 동적으로 삽입해 네트워킹/보안/가시성 로직을 처리 (커널 모듈 없이 커널 내부 동작)

- kube-proxy 완전히 대체 가능 - 서비스 로드밸런싱을 eBPF 해시테이블로 구현
- 네트워크 정책이 IP 기반이 아니라 identity(신원) 기반이며, L3~L7까지 정책 적용 가능
- native routing 또는 overlay 모드 모두 지원, 멀티 클러스터 확장 가능
- CNCF Graduated 프로젝트로 서비스 메시, 관측성(Hubble) 등 부가 기능 많음

▼ 참고자료

<https://github.com/containernetworking/cni>

<https://kubernetes.io/ko/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/>

질문

1. Cilium에서 서비스 로드밸런싱을 eBPF 해시테이블로 구현한다는데 구체적으로 어떻게 구현하는건가?

- Service의 `ClusterIP:Port` 를 key로, 뒤에 있는 실제 Pod들의 `IP:Port` 목록을 value로 하는 해시맵(eBPF map)을 커널에 유지한다. 패킷이 지나가거나(또는 애플리케이션이 `connect()` 를 호출하는 시점) eBPF 프로그램이 이 해시맵을 조회해 즉시 실제 백엔드로 변환(NAT)한다.
 - kube-proxy(iptables 모드): 규칙을 순차적으로 매칭 → 서비스 수가 많아지면 느려짐
 - eBPF 해시맵: O(1) 조회 → 서비스 수와 거의 무관하게 빠름
 - Cilium은 `connect()` 시점(소켓 레벨)에서 미리 변환해버려서 매 패킷마다 NAT를 반복할 필요가 없다.

2. 왜 Flannel의 구조가 단순한건가?

- 관심사가 좁음 → 오직 Pod 간 L3 연결성만 책임지고, NetworkPolicy·L7·서비스 로드밸런싱 같은 부가 기능을 아예 다루지 않음
- 백엔드가 VXLAN(캡슐화) 또는 host-gw(정적 라우트) 정도로 선택지가 적음
- Felix 같은 정책 엔진이나 BGP 데몬이 없음 - 노드마다 flanneld만 돌리면 끝
- 결과적으로 기능이 적다는게 설정할 것도 적고 트러블슈팅 포인트도 적다는 말임!

